

Projeto Final – Programação Avançada para Web

SoundList · Implementação backend com Spring Boot
Arquitetura em camadas · Validações · Mapper · DTOs

Contexto

Você foi contratado para desenvolver o backend de um serviço de gerenciamento de playlists musicais chamado SoundList. A aplicação deve expor uma API REST para criar e gerenciar playlists e as músicas que as compõem. O projeto deve seguir boas práticas de arquitetura, validação de dados e separação de responsabilidades.

Requisitos Funcionais

Playlists

- Criar uma nova playlist com nome e descrição
- Listar todas as playlists
- Buscar uma playlist por ID
- Atualizar os dados de uma playlist existente
- Remover uma playlist (e todas as suas músicas em cascata)

Músicas

- Adicionar uma música vinculada a uma playlist existente
- Listar todas as músicas
- Buscar uma música por ID
- Atualizar os dados de uma música
- Remover uma música

Entidades e Atributos

As duas entidades abaixo devem ser mapeadas com JPA. O relacionamento deve ser configurado com cascata para que a remoção de uma playlist apague todas as suas músicas.

Playlist

Atributo	Tipo	Descrição / Restrição
id	Long	Chave primária, gerada automaticamente
name	String	Nome da playlist. Obrigatório
description	String	Descrição da playlist. Opcional

Music (Música)

Atributo	Tipo	Descrição / Restrição
id	Long	Chave primária, gerada automaticamente
title	String	Título da música. Obrigatório
artist	String	Nome do artista ou banda. Obrigatório
genre	String	Gênero musical. Opcional
duration	Integer	Duração em segundos. Obrigatório, valor positivo
playlist	Playlist	Playlist à qual pertence. Relacionamento @ManyToOne

Endpoints

Playlists

Método	Endpoint	Descrição
POST	/api/playlists	Criar playlist
GET	/api/playlists	Listar todas (paginado)
GET	/api/playlists/{id}	Buscar por ID
PUT	/api/playlists/{id}	Atualizar
DELETE	/api/playlists/{id}	Remover

Músicas

Método	Endpoint	Descrição
POST	/api/musics	Adicionar música
GET	/api/musics	Listar todas (paginado)
GET	/api/musics/{id}	Buscar por ID
PUT	/api/musics/{id}	Atualizar
DELETE	/api/musics/{id}	Remover

Arquitetura Obrigatória

O projeto deve obrigatoriamente seguir a separação em camadas abaixo. Nenhuma entidade JPA deve trafegar diretamente nas respostas ou requisições da API.

Camada	Anotação / Classe	Responsabilidade
Model	@Entity	Entidades JPA mapeadas para o banco de dados
Repository	@Repository	Acesso ao banco via Spring Data JPA
Service	@Service	Regras de negócio e orquestração
Controller	@RestController	Camada HTTP: recebe requisições e retorna respostas
DTO	Record / Class	Objetos de transferência para entrada (Request) e saída (Response)

Requisitos Técnicos

- Spring Boot 3.x como base do projeto
- Spring Data JPA com Hibernate para persistência
- Bean Validation: @Valid, @NotBlank, @NotNull, @Positive nos DTOs
- MapStruct ou ModelMapper para conversão entre entidade e DTO
- H2 (desenvolvimento) ou PostgreSQL como banco de dados
- @ControllerAdvice global para tratamento centralizado de erros
- Paginação obrigatória com Pageable nos endpoints GET /api/playlists e GET /api/musics

Validações Mínimas Exigidas

Playlist

- @NotBlank: nome

Música

- @NotBlank: título e artista
- @NotNull e @Positive: duração (em segundos, deve ser maior que zero)
- @NotNull: playlistId — a playlist informada deve existir no banco

Tratamento de Exceções (Obrigatório)

Deve ser implementado um @ControllerAdvice global cobrindo ao menos os seguintes casos:

- 404 Not Found — entidade não encontrada (playlist ou música inexistente)
- 400 Bad Request — erros de validação dos campos do DTO
- 500 Internal Server Error — erros inesperados do servidor

Todas as respostas de erro devem retornar um corpo JSON com mensagem descritiva.

Critérios de Avaliação

- Controller não deve acessar o Repository diretamente — toda lógica passa pelo Service
- DTOs distintos para entrada (Request) e saída (Response) quando necessário
- Uso correto do mapper, sem conversão manual espalhada pelo código
- Status HTTP adequados nas respostas: 201, 200, 204, 400, 404, 500
- Tratamento de erros centralizado com mensagens descritivas em JSON
- Endpoints de listagem retornam Page<T> com suporte a page, size e sort via Pageable

Observações

- O projeto pode ser realizado individualmente ou em grupos de até 3 pessoas.
- A apresentação será dividida em dois momentos: execução de alguns endpoints (1,5 ponto) e perguntas individuais sobre o desenvolvimento do código (1,5 ponto), totalizando 3 pontos.
- A apresentação ocorrerá no dia 17/06.
- O tratamento de exceções é obrigatório. No mínimo devem ser tratados: entidade não encontrada (404), erros de validação (400) e erros internos do servidor (500), todos com mensagens de erro descritivas no corpo da resposta.